

22CA120	DATA STRUCTURES AND APPLICATIONS	CATEGORY	L	T	P	CREDIT
		PC	3	1	0	4

Preamble

This course will cover various data structures and their operations for manipulating them. Students will learn how to organize the data so that, the data can be accessed and updated efficiently using computer programs. This course illustrates the applications of data structures.

Prerequisite

Nil

Course Outcomes

On the successful completion of the course, students will be able to

CO1	Apply the concepts of stack and queue for suitable applications in tradeoff with time and space complexity.	Apply
CO2	Illustrate the operations like insertion, deletion, traversing on the non linear tree data structure	Apply
CO3	Choose appropriate binary and multiway search tree for performing Searching operations, with an understanding of the trade-off between the time and space complexity.	Apply
CO4	Manipulate disjoint sets by performing union, iterative find-set operations	Apply
CO5	Demonstrate the concepts of advanced data structures including heap in various applications	Apply
CO6	Show the avoidance of collisions in the hash tables using collision resolution techniques including open and closed hashing techniques.	Apply

Mapping with Programme Outcomes and Programme Specific Outcomes

COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	S	S	M									
CO2	S	S	M									
CO3.	S	S	M									
CO4.	S	S	M				L					
CO5.	S	S	M				L					
CO6.	L	S	M				L					

S-Strong; M-Medium; L-Low

Assessment Pattern: Cognitive Domain

Cognitive Levels	Continuous Assessment Tests			Terminal Examination
	1	2	3	
Remember	10	10	10	10
Understand	30	30	10	10
Apply	60	60	80	80
Analyze	0	0	0	0
Evaluate	0	0	0	0
Create	0	0	0	0

Course Level Assessment Questions**Course Outcome 1(CO1):**

1. Suppose a circular queue of capacity (n -1) elements is implemented with an array of n elements. Assume that the insertion and deletion operation are carried out using REAR and FRONT as array index variables, respectively. Initially, REAR=FRONT=0. Give the conditions to detect queue full and queue empty?
2. Given a 5 element stack S (from top to bottom:2,4,6,8,10), and an empty queue Q, remove the elements one-by-one from S and insert them into Q, then remove them one-by-one from Q and re-insert them into S. List the elements in S (from top to bottom).
3. Given an array and a singly linked list. Which of these data structures uses more memory space to store the same number of elements? Justify your answer.

Course Outcome 2(CO2):

1. Perform the AVL algorithm for non AVL trees. In each case, count the number of updated links required by the AVL rotation. Given a simple expression tree, consisting of basic binary operators i.e., +,-,*and / and some integers, write an algorithm to evaluate the expression tree.
2. Suppose inorder and preorder traversal of a binary tree: Inorder D, B, H, E, A, I, F, J, C, G Preorder A,B,D,E,H,C,F,I,J,G
3. Construct binary tree. Show the step by step process with suitable algorithm. Given a red-black tree with n elements, how fast can you sort them using the tree?

Course Outcome 3(CO3)

1. Given a B-Tree with H=5, M=10 and L=10, what is the Maximum and Minimum number of values that can be contained in the leafs of the B-Tree. Remember, all internal nodes and leafs must be atleast half full.
2. Given a red-black tree and a key. Check the given key exist or not without recursion.
3. Create a tree for these words S={ab,ba,ca,caa,caaa,baaa} over the alphabet $\Sigma=\{a,b,c\}$.

Course Outcome 4(CO4)

1. Given a Boolean 2D matrix, find the number of islands using disjoint set.
2. Write pseudocode for make-set, find-set, union using singly linked lists and the weighted union rule. Each object x, has:
 - a) Field repr[x] pointing to the representative of the set containing x,

- b) Field `last[x]` pointing to the last object in the list containing `x`,
 - c) field `size[x]` giving the size of the list containing `x`. `Size[x]` and `Last[x]` are correct only when `x` is representative
3. Suppose you have an implementation of union that is “by-size” and an implementation of find that does not use path compression. Give the parent map (or array) that results from the following sequence: `union(1,2)`, `union(3,4)`, `union(3,5)`, `union(1,7)`, `union(3, 12)`, `union(0,9)`, `union(8,10)`, `union(8,9)`, `union(7,4)`, `union(2,9)` where the unions are: a) by size b) by height c) by size, but now with path compression

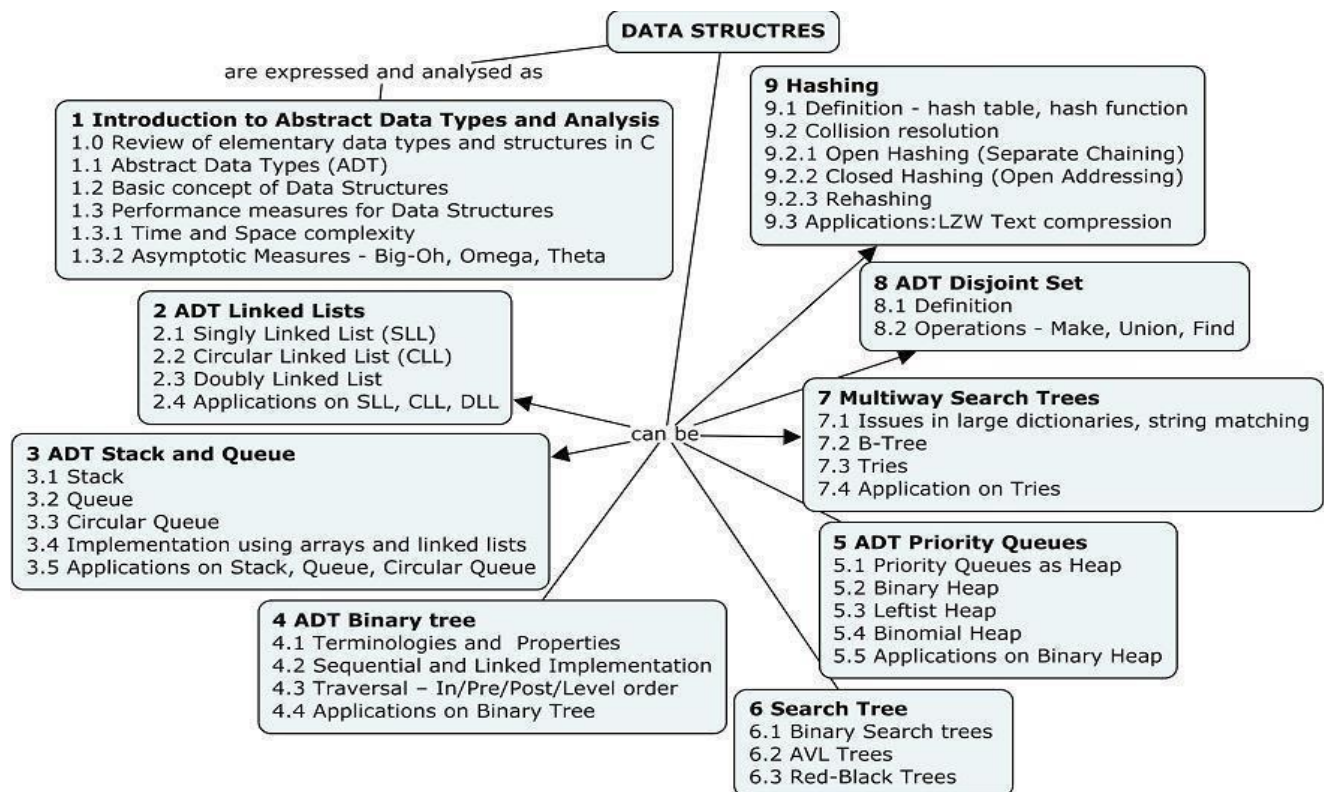
Course Outcome 5(CO5)

1. For a binary heap stored in an array, the root is stored in position 1, the parent of node `i` is stored in position $\text{floor}(i/2)$, the left child is in position `2i`, and the right child is in position `2i+1`. What about a d-heap stored in an array? In what positions are the children and parent of node `i` stored? [Hint: to start, assume that the root is at position 0. Then modify your results to work with the root at position 1]
2. Show the result of inserting keys 1 to 15 in order (i.e. 1 first, then 2 second, then 3 third, etc.) into an initially empty left ist heap. Use the left ist heap insert (i.e. merge) algorithm a teach step. Show each step for this process.
3. Prove or disprove: A perfectly balanced tree forms if keys 1 to $2k-1$ are inserted in order (again this means 1 first, then 2 etc) in to an initially empty left ist heap. K is a positive integer.

Course Outcome 6(CO6):

1. Given input {4371, 1323, 6173, 4199, 4344, 9679, 1989} and a hash function $h(x) = x(\text{mod})10$, show the resulting
 - a. Separate chaining hash table
 - b. Hash table using linear probing, quadratic probing
2. Consider implementing a hash table for an application in which we will build an initial hash table by inserting a substantial collection of records. After this, we expect that the number of insertions and the number of deletions performed to be roughly the same, although there may be long runs of consecutive insertions or consecutive deletions. Furthermore, the table will use a probe strategy to resolve any collisions that occur during insertion, and therefore we will “tombstone” cells from which a record has been deleted. If we implement the hash table described above, then when we search for a record, we cannot conclude the record is not in the table until we have found an empty cell in the table, not just a tombstone. (We will ensure that the table never reaches the state that there are no empty cells.) Explain carefully why the search cannot stop when a tombstone is encountered.
3. Let $m=17$, $h_1(x)=(k+15)\%m$, $h_2(x)=(4k+11)\%m$, and $h_3(x)=(7k+2)\%m$. Insert the keys 23, 7, 50, and 91 into the bit vector, and show the resulting vectors content. Then, find a key that is a false positive; that is, find a key that appears to have been inserted, but wasn't.

Concept Map



Syllabus

Introduction to Abstract Data Types and Analysis: Review of elementary data types and structures in C - Abstract Data Types (ADT) - Basic concept of Data Structures - Performance measures for Data Structures - Time and Space Complexity Asymptotic Measures

- Big - Oh, Omega, Theta. **Linked Lists:** Singly Linked List(SLL) - Definition and Operations (Create, Insert, Delete, Search, Reverse) - Circular Linked List (CLL) - Definition and Operations (Create, Insert, Delete, Search) - Doubly Linked List - Definition and Operations (Create, Insert, Delete, Search) - Applications on Lists- SLL: Representing univariate polynomial and adding two univariate polynomial - CLL: Josephus Problem - DLL: Checking Palindrome, Quick Sort. **Stack and Queue:** Stack - Definition and Operations (Create, Push, Pop), Queue - Definition and Operations (Create, Enqueue, Dequeue), Circular Queue - Definition and Operations (Create, Enqueue, Dequeue), Implementation using arrays and linked lists Applications - Stack: Arithmetic Expression Evaluation - Queue: First Come First Serve (FCFS) Scheduling - Circular Queue: Round Robin Scheduling. **Binary tree:** Terminologies and Properties - Sequential and Linked Implementation - Traversal - Inorder, Preorder, Postorder, Levelorder - Application - Huffman coding and expression trees. **Priority Queues:** Priority Queues as Heap, Heap Implementation, Binary Heap - Definition and Operations (Create, Insert, Delete), Leftist Heap - Definition and Operations (Create, Insert, Delete), Binomial Heap - Definition and Operations (Create, Insert, Delete), Applications on Binary Heap: Heap Sort, use of winner trees in merge sort as an external sorting algorithm. **Search Tree:** Binary Search trees - Definition, Properties and Operations (Create, Insert, Delete, Search), AVL Trees -

Definition, Importance of Balancing, Properties and Operations (Create, Insert, Delete, Search), Red-Black Trees-Definition, compare with AVL, Properties and Operations (Create, Insert, Delete, Search). **Multiway Search Trees:** Issues in large dictionaries, string matching, B-Tree-Definition, Properties and Operations (Create, Insert, Delete, Search), Tries-Definition and Operations (Create, Insert, Delete, Search), Application on Tries: Pattern Searching. **Disjoint Set:** Definition and Operations (Make, Union, Find). **Hashing:** Definition - hash table, hash function, Collision resolution and overflow handling techniques, Open Hashing (Separate Chaining), Closed Hashing (Open Addressing)-Linear, Quadratic, Double, Rehashing, Applications - LZW Text compression algorithm.

Learning Resources

1. Mark Allen Weiss, "Data Structures and Algorithm Analysis in C++", Pearson, 2007
2. Adam Drozdek, "Data structures and Algorithms in C++", Cengage Learning; 4th Edition, 2012.
3. Y. Langsam, M.J. Augenstein and A.N. Tanenbaum, "Data Structure Using C and C++", Pearson Education, 2nd Edition, 2015.

Course Contents and Lecture Schedule

Module No.	Topic	No. of Lectures
1	Introduction to Abstract Data Types and Analysis	
1.0	Review of elementary data types and structures in C	1
1.1	Abstract Data Types (ADT)- Basic concept of Data Structures	1
1.3	Performance measures for Data Structures: Time and Space complexity - Asymptotic Measures: Big-Oh, Omega, Theta	2
2	Linked Lists	
2.1	Singly Linked List (SLL)-Definition and Operations (Create, Insert, Delete, Search, Reverse)	2
2.2	Circular Linked List (CLL)-Definition and Operations (Create, Insert, Delete, Search)	2
2.3	Doubly Linked List – Definition and Operations (Create, Insert, Delete, Search)	1
2.4	Applications on Lists	1
2.4.1	SLL: Representing univariate polynomial and adding two univariate polynomial	1
2.4.2	CLL: Josephus Problem	1
2.4.3	DLL: Checking Palindrome, Quick Sort	1
3	Stack and Queue	
3.1	Stack, Queue, Circular Queue- Definition and Operations (Create, Insert, Delete)	2
3.2	Implementation using arrays and linked lists	1
3.3	Applications	1
3.3.1	Stack: Arithmetic Expression Evaluation	2
3.3.2	Queue : First Come First Serve (FCFS) Scheduling	1
3.3.3	Circular Queue:- Round Robin Scheduling	1
4	Binary tree	

4.1	Terminologies and Properties-Sequential and Linked Implementation	2
4.2	Traversal - Inorder, Preorder, Postorder, Levelorder	1
4.3	Application-Huffman coding and expression trees	1
5	Priority Queue	
5.1	Priority Queue as Heap, Heap Implementation	1
5.2	Binary Heap-Definition and Operations (Create, Insert, Delete)	1
5.3	Left ist Heap-Definition and Operations (Create,Insert,Delete)	1
5.4	Binomial Heap-Definition and Operations (Create,Insert,Delete)	1
5.5	Applications on Binary Heap: Heap Sort, use of winner trees in merge sort as an external sorting algorithm	1
6	Search Tree	
6.1	Binary Search trees - Definition, Properties and Operations(Create,Insert,Delete,Search)	2
6.2	AVL Trees-Definition, Importance of Balancing, Properties and Operations (Create, Insert, Delete,Search)	1
6.3	Red-Black Trees - Definition, Compare with AVL ,Properties and Operations (Create, Insert, Delete,Search)	1
7	Multiway Search Trees	
7.1	Issues in large dictionaries, string matching	1
7.2	B-Tree-Definition, Properties and Operations(Create,Insert,Delete,Search)	1
7.3	Tries-Definition and Operations (Create,Insert,Delete,Search)	1
7.4	Application on Tries: Pattern Searching	1
8	Disjoint Set	
8.1	Definition and Operations(Make, Union, Find)	1
9	Hashing	
9.1	Definition-hash table, hash function	1
9.2	Collision resolution and overflow handling techniques	1
9.2.1	Open Hashing(Separate Chaining)	1
9.2.2	Closed Hashing (Open Addressing) - Linear, Quadratic, Double	1
9.2.3	Rehashing	1
9.3	Applications-LZWT ext compression algorithm.	1
Total No of Hours		45

Course Designers:

1. P.Sharmila

psaca@tce.edu